

TypeScript

Introduction:

TypeScript is a strongly typed programming language that builds on JavaScript.

It uses compile-time type checking.

JavaScript code is generated from TypeScript.

As a result, TypeScript can be used anywhere JavaScript is used, including:

Frontend

Backend

Nature of TypeScript

TypeScript is an object-oriented programming language.

It is a case-sensitive language.

TypeScript Compiler

TypeScript has an official compiler.

It can be installed using npm.

In Angular:

The Ahead-of-Time (AOT) compiler converts Angular HTML (forms) and TypeScript code into efficient JavaScript.

TypeScript – Simple Data Types

Primitive Types

TypeScript supports several simple (primitive) data types.

Main Primitive Types (JavaScript & TypeScript)

There are three main primitives:

boolean: Represents true or false values

number: Represents whole numbers and floating-point values

string: Represents text values

Additional Primitive Types (Later Versions):

bigint: Used for very large numbers. Supports larger positive and negative values than the number type

symbol: Used to create a globally unique identifier. Symbols are Unique, Immutable.

Uses of symbol:

Creating property keys for objects

Creating private properties or methods in classes

TypeScript – Variable Declaration & Syntax:

Syntax:

<variable> : <datatype> = value

Example

```
let a: number = 10;
```

Example Program (app.ts)

```
let a = 10;

console.log(a);

console.log(typeof(a));

var s = "hyderabad";

console.log(s);

console.log(typeof(s));

const b = 45.66;

console.log(b);
```

```
console.log(typeof(b));  
  
let c = true;  
  
console.log(c);  
  
console.log(typeof(c));
```

Explanation of Output

a → number

s → string

b → number

c → boolean

The typeof() operator is used to check the data type of a variable at runtime.

Variable Declarations in TypeScript / JavaScript

var: Declares variables with function scope or global scope

Allows:

Re-declaration

Re-assignment

Not recommended in modern TypeScript due to scope issues

Example

```
var x = 10;  
  
var x = 20; // allowed
```

let: Declares variables with block scope

Allows:

Re-assignment

Does not allow re-declaration in the same block

Example

```
let y = 10;
```

```
y = 20; // allowed
```

```
// let y = 30; ✗ not allowed
```

const: Declares block-scoped constants

Allows

Does not allow re-assignment after initial assignment

Value must be assigned at declaration time

Example

```
const z = 50;
```

```
// z = 60; ✗ not allowed
```

Comparison Summary (Exam-Friendly)

Keyword	Scope	Re-declaration	Re-assignment
var	Function / Global	✓ Yes	✓ Yes
let	Block	✗ No	✓ Yes
const	Block	✗ No	✗ No

Tip for Exams

Prefer let and const and Avoid var

Use const when the value should not change

TypeScript – Example 2 (Typed Variables)

Example Code

```
let a: number = 10;  
console.log("a value:" + a);
```

```
var s: string = "hyderabad";  
console.log("s value:" + s);  
const b: number = 45.66;  
console.log("b value:" + b);  
let c: boolean = true;  
console.log("c value:" + c);
```

Explanation

Variables are declared with explicit data types

Values are printed using console.log()

String concatenation (+) is used to display labels with values

Note: Viewing Output in Browser

To open the console in Google Chrome:

Click on Three dots (:)

Go to Tools → More Tools

Select Developer Tools

Open the Console tab

Special Types in TypeScript

any Type:

any is a special type that disables type checking.

Allows assigning values of any data type.

Use with caution, as it removes TypeScript's safety benefits

Example 3: any Type

```
let a: any;  
  
a = 10;  
  
console.log(a);  
  
a = "hyd";  
  
console.log(a);  
  
a = 56.77;  
  
console.log(a);
```

Explanation

The same variable holds Number, String, Decimal value. No type error occurs because the variable is declared as any.

Math Functions Example

Example 4: Math.round()

```
console.log(Math.round(45.67));
```

Output

46

Explanation

`Math.round()` rounds a number to the nearest integer

Values $\geq .5$ are rounded up

Values $< .5$ are rounded down

Exam Tips ✦

Prefer specific types (number, string, boolean)

Avoid any unless absolutely necessary

`Math.round()` returns a number

TypeScript – Example 5 (Type Safety vs any)

Example 5: Type Error Case

```
let a = true;
```

```
a = "hyd";
```

```
Math.round(a);
```

Error

Type 'string' is not assignable to type 'boolean'

Explanation

Variable `a` is initially assigned a boolean value

TypeScript infers the type of `a` as boolean

Assigning a string ("hyd") to `a` causes a compile-time error

This demonstrates TypeScript's strict type checking

Example Using any Type

```
let a: any = true;
```

```
a = "hyd";
```

```
console.log(Math.round(a));
```

Output

NaN

Explanation

a is declared as type any, so:

Assigning a string does not cause a compile-time error

Math.round() expects a number

Since "hyd" is not numeric, the result is:

NaN (Not a Number)

Key Concepts Highlighted (Exam Important)

Type Inference

TypeScript automatically determines the type of a variable based on its initial value

Type Safety

TypeScript prevents assigning incompatible types

Errors are caught at compile time, not runtime

any Type – Warning ⚠

Disables type checking

Can lead to runtime errors

Should be avoided unless absolutely necessary

Exam Tips ✨

TypeScript enforces strong typing

Using any removes compile-time safety

Math.round() works only with numbers

Invalid numeric operations result in NaN

Functions in TypeScript

What is a Function?

A function is used to execute a set of statements as a single block.

Functions help in code reusability.

A function:

- Can have a return type

- Can return a value

- Can be called any number of times

Example 1: Creating a Function Without Arguments

Function Definition

```
function fun1() {  
    console.log("Sathya Technologies");  
}
```

Function Call

```
fun1();
```

Explanation

function fun1() defines a function named fun1

No arguments (parameters) are passed

console.log() prints the message

Calling fun1() executes the function body

Exam Tips ✨

Functions improve maintainability and reusability

A function can be:

- With arguments

- Without arguments

- A function must be called to execute its code

Functions in TypeScript – Return Type

Example 2: Function Without Arguments and With Return Type

Function Definition

```
function fun1(): void {  
    console.log("Sathya Technologies");  
}
```

Function Call

```
fun1();
```

Explanation

() : void specifies the return type of the function

void means:

The function does not return any value

The function takes no arguments

Only performs an action (console.log)

Key Points (Exam Important)

Every function in TypeScript can have a return type

void is used when no value is returned

If a return type is not specified TypeScript infers it automatically

Exam Tips 🚀

Use void for functions that:

Print output

Perform actions

Always specify return types for better readability and safety

Ex: 3 Creating function with 2 arguments

```
function sum(a:number, b:number):number {  
    return a+b;  
}
```

Ex: 4 Optional parameters

```
function add(a:number, b:number, c?:number){  
    return a+b+(c | 0);  
}
```

```
}
```

Here, the **?** makes the argument **c** as optional. If **c** value is passed, it will be added to the result. If not 0 will be added.

Ex: 5 Defaults Parameters

```
function add(a:number=5, b:number=6){  
    return a+b;  
}
```

Ex: 6 Anonynouse function

```
let s = function(a:number, b:nuber){  
    return a+b;  
}  
  
s(2,3);
```

This is an anonymouse function. Arrow functions also considered anonymouse functions.

Callback functions:

A function that is used as an argument for another function or returned by another function is called as Callback Function. It will be executed when the main function is executed.

Ex:

```
function average(arr:number[], cb){
    let sum = arr.reduce((total, num) => total+num, 0);
    return cb(sum, arr.length);
}

average([1,2,3,4,5,6,7,8,9,10], (total,length):number => total/length);
```

Class and Object:

Class:

Class is a template or prototype or blueprint which is used to create a new user defined datatype. Class contains variables and methods. Variables are used to store the values and methods used to perform operations.

Object:

Object is created using a class. It is called as class variable or instance variable or instance copy of the class. It contains the memory address or location of the class. Object is itself a class because, it refers to a class.

Access Modifiers:

There are 3 access modifiers in TypeScript:

i. Public:

The variables and methods with public access modifier will be accessed by any class and anywhere. They are open to use and modify as we wish.

ii. Private:

The variables and methods with private will be accessed within the same class only. Outside this class these values are not accessible.

iii. Protected:

The variables with protected will be accessed in the same class as well as the class which inherits this class.

